

0.93.3. Un jeu de bombes — la bombe explose

Le 0.93.2, et la bombe explose au bout de 2 secondes : des flammes - et | sur sa case et une case autour, sauf dans les X. Une demi-seconde plus tard, elles s'éteignent.

1. Ce qu'il faut comprendre

C'est le 0.93.2, avec une chose en plus : l'explosion. Deux secondes après la pose, la B disparaît et des **flammes** apparaissent : sur sa case, et **une case** à droite, à gauche, en bas, en haut. Une demi-seconde après, elles s'éteignent.

Les flammes sont des signes : `-` pour les flammes couchées (à gauche, à droite, et le centre), `|` pour les flammes debout (en haut, en bas). `texte(c, 1, "-")` les écrit, comme un mot d'une lettre.

Compter le temps : `bdelai` compte les images depuis la pose (`bdelai = 0;` quand on pose). À chaque image : `bdelai = bdelai + 1;`. `image()` revient 60 fois par seconde : à **120**, deux secondes ont passé, la bombe explose. Les flammes, elles, comptent dans `fdelai` jusqu'à **30** : une demi-seconde.

Au moment d'exploser : `bombe = 0;` (plus de bombe), le centre de l'explosion est rangé dans `fx` et `fy`, `feu = 1;` (les flammes sont là), et `flammes(1);` les dessine.

Ce qui est nouveau ici : une fonction qui dessine OU efface. `flammes(allume)` passe sur les cases de l'explosion ; pour chacune, elle appelle `caseFeu(c, 1, allume, debout)`. Si `allume` vaut 1, `caseFeu` écrit la flamme ; s'il vaut 0, elle efface la case. **Pourquoi une seule fonction ?** Pour être sûr que `flammes(0)` efface **exactement** les cases que `flammes(1)` a dessinées. Des fonctions avec des paramètres : le 0.33.

Le X arrête la flamme : avant chaque case autour du centre, `if (lire(fx + 1, fy) != ALPHABET[23])`. Un pilier ou le cadre ne brûle pas. Le centre, lui, est toujours brûlé : c'est la case de la bombe.

debout : le 4e paramètre de `caseFeu`. 1 : `|` ; 0 : `-`. En bas et en haut, on passe 1 ; au centre, à droite et à gauche, 0.

Deux précautions : 1. On ne pose pas de nouvelle bombe tant que les flammes sont là (`&& feu == 0`) : sinon `fx` et `fy` changeraient, et `flammes(0)` effacerait au mauvais endroit. 2. Après `flammes(0)`, on **redessine le O** : s'il était resté dans une flamme, l'effacement l'a effacé aussi.

Le O ne marche pas dans les flammes : elles ne sont pas une case vide (le 0.93.2). Pour l'instant, elles ne lui font rien s'il est dessus ; au 0.93.8, elles le feront perdre.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1191 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      87 o ← la console, à l'allumage – la mise en route de la console
chaque_minuteur() 124 o ← main() – une fonction fournie par le langage
caseFeu()       71 o ← flammes() – ta fonction
flammes()       173 o ← main() – ta fonction
main()          440 o ← démarrage – ta fonction
VBlank          8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage   28 o ← image() – attend l'image suivante
AttendreVBlank  16 o ← démarrage – attend que l'écran soit libre
AttendreAcces   12 o ← EffacerCases, EcrireTexte – attend de pouvoir écrire une case
AdresseCase     10 o ← effacer(), texte(), lire(), poser() – trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() – efface des cases
EcrireTexte     10 o ← texte() – écrit un texte, lettre par lettre
CopierTuiles    67 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage – vide les deux cartes de l'écran
LireManette     37 o ← bouton() – lit les huit boutons
Tuiles          80 o ← CopierTuiles – 5 lettres sur 43
"|"            1 o ← texte() – un texte gravé
"_"            1 o ← texte() – un texte gravé
```

Lettres gravées : BOX-|

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser()        x 6   78 o sur place, dans main() + AdresseCase (10 o)
lire()         x 5   63 o sur place, dans flammes(), main() + AdresseCase (10 o)
texte()        x 2   36 o sur place, dans caseFeu() + AdresseCase (10 o), EcrireTexte (10 o)
effacer()      x 2   30 o sur place, dans caseFeu(), main() + AdresseCase (10 o), EffacerCases (12 o)
bouton()       x 5   25 o sur place, dans main() + LireManette (37 o)
image()        x 1    3 o sur place, dans main() + AttendreImage (28 o)
images()       x 1    2 o sur place, dans chaque_minuteur()
```

Tes fonctions appelées :

```
caseFeu()      x 5  100 o sur place, dans flammes()
flammes()      x 2   13 o sur place, dans main()
chaque_minuteur() x 1  10 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UN JEU DE BOMBES ----
// Les lettres : X = un mur, O = le joueur, B = une bombe, W = un ennemi.
// ALPHABET[23] : le X   ALPHABET[14] : le O
// ALPHABET[1]  : le B   ALPHABET[22] : le W

#include <effacer> // efface des cases, ou tout le fond
#include <texte>   // écrit un texte à l'écran
#include <lire>    // lit la tuile posée sur une case
#include <ALPHABET> // les lettres de la police : ALPHABET[0] est le A
#include <poser>   // pose une tuile sur une case du fond
#include <chaque>  // répond 1 toutes les n millisecondes
#include <bouton>  // lit un bouton de la manette
```

2. UNE VARIABLE — OX

```
uint8_t ox = 1; // le O, le joueur : sa colonne... (le 0.93.1)
```

3. UNE VARIABLE — OY

```
uint8_t oy = 1;          // ...et sa ligne. Il part du coin (1, 1).
```

4. UNE VARIABLE — NX

```
uint8_t nx = 0;          // la case d'ARRIVÉE, calculée AVANT de bouger (le 0.82)
```

5. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

6. UNE VARIABLE — BOMBE

```
uint8_t bombe = 0;       // 1 : une bombe est posée ; 0 : pas de bombe (le 0.93.2)
```

7. UNE VARIABLE — BX

```
uint8_t bx = 0;          // la place de la bombe : sa colonne...
```

8. UNE VARIABLE — BY

```
uint8_t by = 0;          // ...et sa ligne
```

9. UNE VARIABLE — BDELA

```
uint8_t bdelai = 0;       // NOUVEAU : les images passées depuis la pose de la bombe
```

10. UNE VARIABLE — FEU

```
uint8_t feu = 0;          // 1 : les flammes sont à l'écran ; 0 : non
```

11. UNE VARIABLE — FDELA

```
uint8_t fdelai = 0;       // les images passées depuis l'explosion
```

12. UNE VARIABLE — FX

```
uint8_t fx = 0;          // le centre de l'explosion : sa colonne...
```

13. UNE VARIABLE — FY

```
uint8_t fy = 0;          // ...et sa ligne

// ---- NOUVEAU : UNE case de flamme ----
// allume = 1 : on dessine la flamme ; allume = 0 : on l'efface.
// debout = 1 : une flamme verticale, le | ; debout = 0 : horizontale, le -.
```

14. UNE FONCTION — CASEFEU()

```
void caseFeu(uint8_t c, uint8_t l, uint8_t allume, uint8_t debout) {
    if (allume == 0) {
        effacer(c, l, 1);           // la flamme s'éteint : la case est vide
    } else {
        if (debout == 1) {
            texte(c, l, "|");
        } else {
            texte(c, l, "-");
        }
    }
}

// ---- NOUVEAU : toute l'explosion, le centre et une case de chaque côté ----
//
//      |                le centre : là où était la B
//      - - -           une case à droite, à gauche, en bas, en haut...
//      |                ...sauf si c'est un X : le mur arrête la flamme
//
// flammes(1) dessine, flammes(0) efface : les MÊMES cases, forcément.
```

15. UNE FONCTION — FLAMMES()

```
void flammes(uint8_t allume) {
    caseFeu(fx, fy, allume, 0);           // le centre
    if (lire(fx + 1, fy) != ALPHABET[23]) { // à droite, si ce n'est pas un X
        caseFeu(fx + 1, fy, allume, 0);
    }
    if (lire(fx - 1, fy) != ALPHABET[23]) { // à gauche
        caseFeu(fx - 1, fy, allume, 0);
    }
    if (lire(fx, fy + 1) != ALPHABET[23]) { // en bas : une flamme debout
        caseFeu(fx, fy + 1, allume, 1);
    }
    if (lire(fx, fy - 1) != ALPHABET[23]) { // en haut
        caseFeu(fx, fy - 1, allume, 1);
    }
}
```

16. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // ---- Le terrain : le cadre et les piliers X (le 0.93) ----
    for (uint8_t l = 0; l < 17; l++) {
        for (uint8_t c = 0; c < 19; c++) {
            if (l == 0 || l == 16 || c == 0 || c == 18) {    // le cadre
                poser(c, l, ALPHABET[23]);
            }
            if (c % 2 == 0 && l % 2 == 0) {                  // les piliers : colonne ET ligne paires
                poser(c, l, ALPHABET[23]);
            }
        }
    }

    poser(ox, oy, ALPHABET[14]);                          // le 0 à sa place (le 0.93)

    while (true) {
        image();

        // ---- le 0 bouge avec la croix, un pas toutes les 150 ms (le 0.93.1) ----
        if (chaque(150)) {
            // 1. la case d'ARRIVÉE : on part de la case actuelle (le 0.82).
            //     else if : une seule flèche à la fois, pas de pas en diagonale.
            nx = ox;
            ny = oy;
            if (bouton(DROITE)) {
                nx = ox + 1;
            } else if (bouton(GAUCHE)) {
                nx = ox - 1;
            } else if (bouton(BAS)) {
                ny = oy + 1;
            } else if (bouton(HAUT)) {
                ny = oy - 1;
            }

            // 2. une flèche est tenue (l'arrivée n'est pas la case où l'on est)...
            if (nx != ox || ny != oy) {
                // 3. ...et l'arrivée est VIDE : ni X, ni B, ni flamme. (le 0.93.2)
                //     lire() rend 0 pour une case vide : c'est l'espace, la tuile 0.
                if (lire(nx, ny) == 0) {
                    effacer(ox, oy, 1);                    // le 0 quitte sa case...
                    if (bombe == 1 && ox == bx && oy == by) {
                        poser(bx, by, ALPHABET[1]);        // ...mais s'il était sur la bombe, la B apparaît (le 0.93.2)
                    }
                    ox = nx;                                // ...et va sur la case d'arrivée
                    oy = ny;
                    poser(ox, oy, ALPHABET[14]);
                }
            }
        }

        // ---- A : poser une bombe, là où est le 0 (le 0.93.2) ----
        // bombe == 0 : une seule bombe à la fois.
        // feu == 0 : pas pendant les flammes (le 0.93.3).
        if (bouton(A) && bombe == 0 && feu == 0) {
            bombe = 1;
            bx = ox;                                        // la bombe est sous le 0
            by = oy;
            bdelai = 0;                                    // on commence à compter (le 0.93.3)
        }

        // ---- NOUVEAU : la bombe explose au bout de 2 secondes ----
        // image() revient 60 fois par seconde : 120 images, 2 secondes.
        if (bombe == 1) {
            bdelai = bdelai + 1;
        }
    }
}
```

```

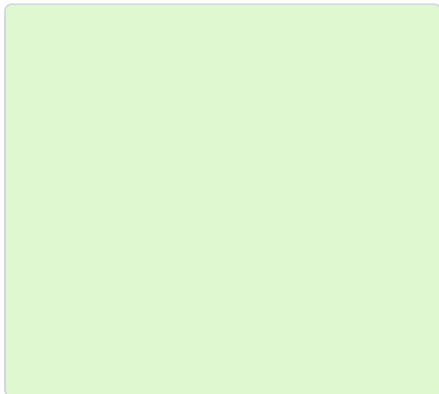
    if (bdelai == 120) {
        bombe = 0;                                // plus de bombe...
        fx = bx;                                  // ...elle explose, là où elle était
        fy = by;
        feu = 1;
        fdelai = 0;
        flammes(1);                                // on dessine les flammes
    }
}

// ---- NOUVEAU : les flammes s'éteignent au bout d'une demi-seconde (30 images) ----
if (feu == 1) {
    fdelai = fdelai + 1;
    if (fdelai == 30) {
        feu = 0;
        flammes(0);                                // on efface les MÊMES cases
        poser(ox, oy, ALPHABET[14]);              // le 0 était peut-être dans une flamme : on le remet
    }
}
}
}

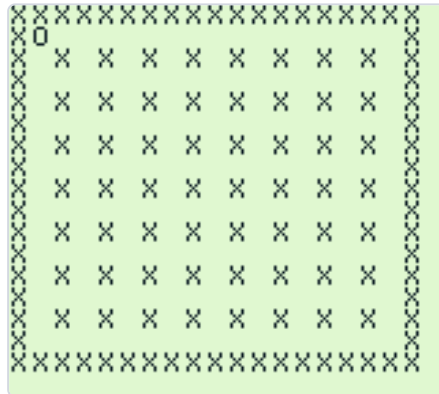
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



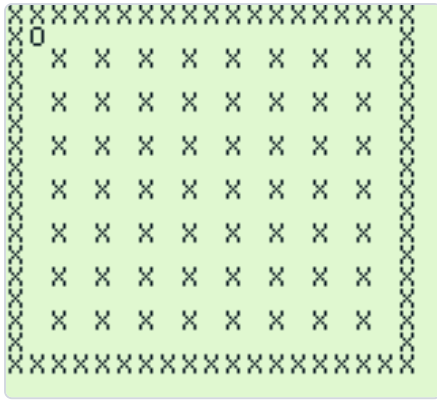
la toute première image — la console vient de démarrer — image 1



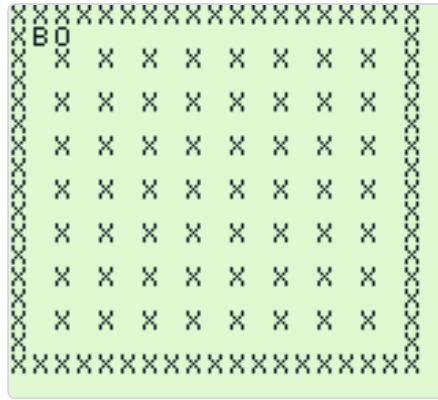
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

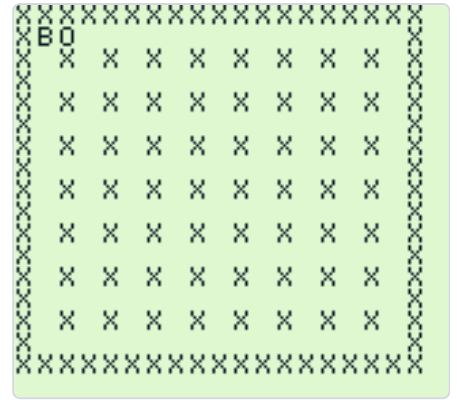
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



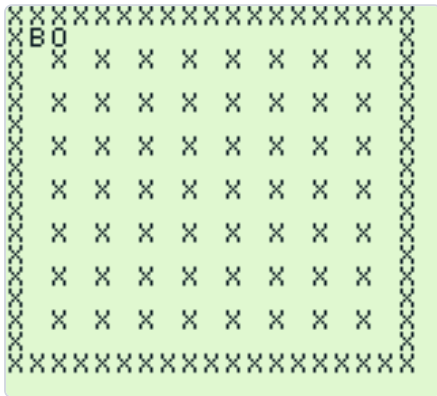
A — l'écran n'a pas bougé



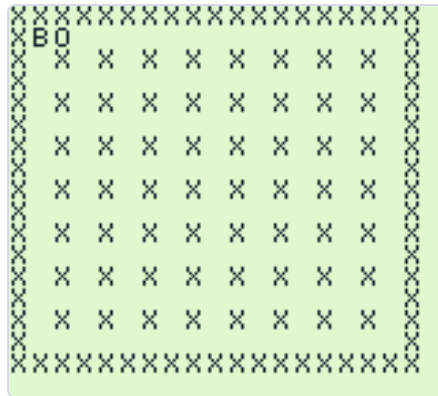
DROITE — l'écran a changé



GAUCHE — l'écran n'a pas bougé



HAUT — l'écran n'a pas bougé



BAS — l'écran n'a pas bougé

5. Ce qu'on doit voir

A, puis on s'écarte : deux secondes après, la bombe explose en croix (- et |), puis les flammes s'éteignent.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ l'explosion : - en (1, 1) et (2, 1), | en (1, 2)
- ✓ les X du cadre ne brûlent pas
- ✓ les flammes s'éteignent, le O est toujours là

Ensuite : **0.93.4. Un jeu de bombes — des flammes plus longues** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++